# CS39440 Software Development Methodologies Report: A Web-Based Restaurant Ordering System

G401 Computer Science (with integrated year in industry)
Aberystwyth University

**Student:**
Ethan Swain (ets6)
**Supervisor:**
Emmanuel Isibor (emi3)

7th March 2026

Version 1.0 (Release)

# The Waterfall Model

The Waterfall Model is a linear and sequential approach to software development. It has six distinct phases (requirements, design, implementation, test, deployment, and maintenance) which must be completed before the next one begins, and there must be no overlapping.

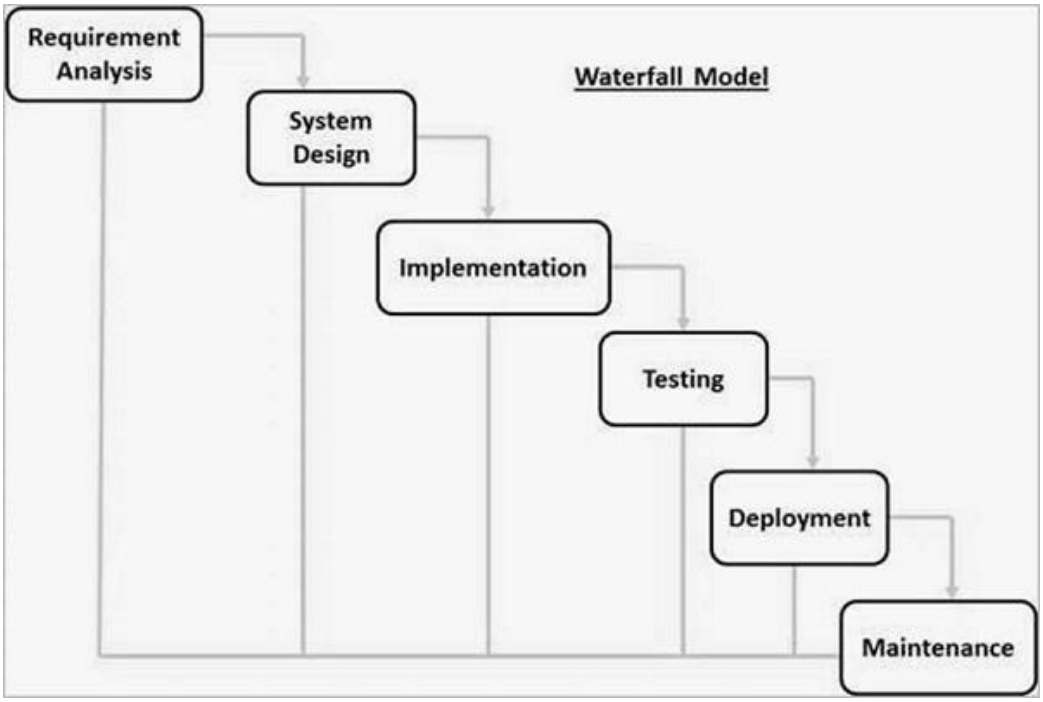


**Figure 1:** The Waterfall Model (TutorialsPoint).

## Advantages

- **Clear Project Structure:** Each stage has well-defined deliverables and review processes which make it very easy to manage and leaves no room for confusion. There is a clear end goal that you are working towards.
- **Extensive Documentation:** The project is highly documented as both the design and requirements are finalised before coding starts. This makes it great for handovers.
- **Set Costs:** The planning ensures that the time and cost of the project are known upfront.
- **Increased Responsibility and Accountability:** A team takes responsibility for each process phase. Each phase has a clear timeline, set of goals, and milestones to reach.

## Disadvantages

- **Limited Flexibility:** Once a phase is completed, it is difficult and costly to go back and change previous decisions if the requirements change or if there is an issue.
- **Late Testing:** If a large issue or design flaw is found during testing, then there could be lots of implications in trying to fix this as the testing phase happens late into the lifecycle.
- **Limited Opportunities for Client Feedback:** After the requirement phase is completed, the client can't really change anything about the project. The client may request lots of changes to the project after it launches, such as additions of new features to the existing code.
- **Longer Delivery Times:** Because it follows an inflexible step-by-step process, the delivery of the final product could take longer.

# Agile Methodology

Agile is an iterative and incremental process, it divides work into phases (called “sprints”) and emphasises continuous delivery and improvement. A phase is usually 1-4 weeks long. The goal is to produce a working piece of software by the end of each sprint, and then continuously refining it based on feedback from the client. Unlike the Waterfall Model where each team works on a phase separately, Agile emphasises teamwork, open communication, and adaptability by using cross-functional teams.

**Figure 2:** The Agile Methodology (Jayathilaka, C).

## Advantages

- **Highly Adaptable:** Any changes to requirements can easily be incorporated at the start of a new sprint, even late in development.
- **Continuous Feedback:** Stakeholders can ensure that project is following what they want as they can see working software early on.
- **Greater Team Collaboration:** There is daily communication and shared goals between the team, resulting in a more motivated and productive environment.
- **Improved Product Quality:** By holding retrospectives and continuously testing, bugs can be identified early on.

## Disadvantages

- **Scope Creep:** It is easy for the project to expand beyond its original timeframe if new features are constantly added.
- **Unpredictable Costs and Timelines:** It can be harder to predict the exact final cost and delivery date when starting the project.
- **Minimal Documentation:** Focusing on working software can lead to insufficient documentation, making it harder for handovers and causing issues for future maintenance.
- **High Resource/Time Demand:** It requires an active involvement from both stakeholders and developers.

# Kanban Methodology

Kanban is an Agile framework that focuses heavily on continuous workflow rather than time-boxed sprints. Teams use a visual Kanban board (with columns like “To Do”, “In Progress”, and “Done”) to track tasks, identify bottlenecks, and optimise delivery cycles. A key practice is to set Work In Progress (WIP) limits to prevent burnout and identify any bottlenecks.

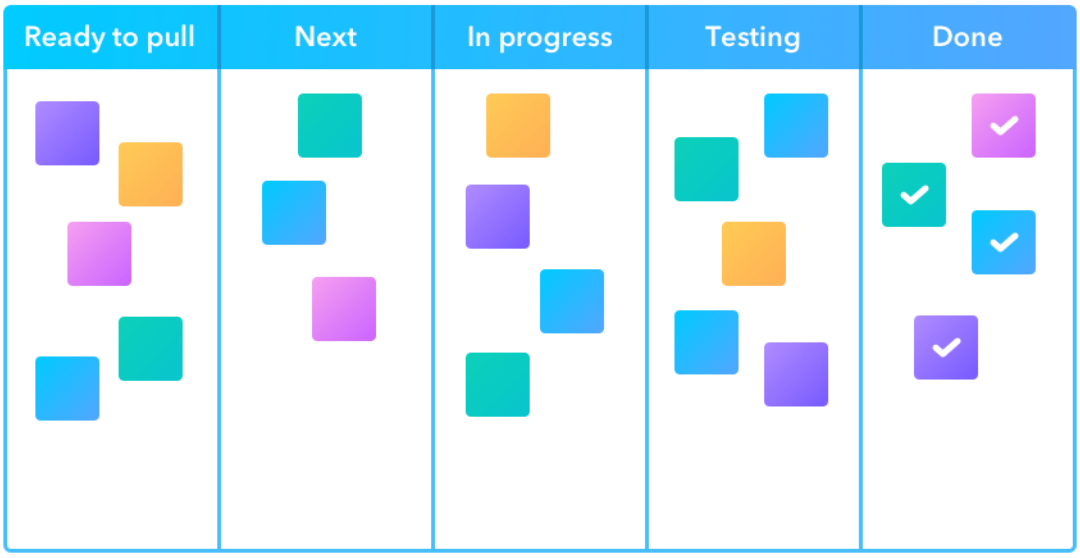


**Figure 3:** The Kanban Methodology (Infinity).

## Advantages

- **Increased Transparency:** Everyone can see where a task is in the pipeline at any given moment.
- **Immediate Bottleneck Identification:** If there are too many tasks in the “Testing” column, then the team immediately knows where the workflow is blocked.
- **Improved Flexibility:** Priorities can be changed quickly, and new tasks can be added as soon as the capacity allows.
- **Increased Productivity:** Limiting the amount of WIP items prevents multitasking and ensures tasks are completed faster.

## Disadvantages

- **Lack of Deadlines:** As there are no fixed sprint iterations, projects may lack urgency and lead to potential delays.
- **Board Complexity:** If the board is not maintained properly, then it can become too complex, confusing, or outdated, worsening efficiency.
- **Requires High Discipline:** Team members must consistently update the board for it to be effective.

# The Spiral Model

The Spiral Model combines the flexible, iterative nature of Agile with the structured approach of Waterfall. It has a heavy emphasis on risk analysis and is mainly used in large and complex projects. It continuously loops through four phases (planning, risk analysis, engineering, and evaluation).

**Figure 4:** The Spiral Model (GeeksforGeeks).

## Advantages

- **Good Risk Management:** Risks are identified and mitigated very early into the project lifecycle.
- **Good for Complex Projects:** It is ideal for large, high-budget systems or mission-critical systems where there is no room for failure.
- **Flexibility:** It allows for changes to be introduced, even in later stages.
- **Early Prototyping:** Prototypes are used in each cycle which allows the user to visualise the system early on.

## Disadvantages

- **Highly Complex:** It is often overkill from smaller/medium-sized projects.
- **High Cost and Time:** It is generally expensive and time-consuming, which again makes it unsuitable for small, low-risk projects.
- **Relies on Team Expertise:** The team must be able to accurately identify and assess risks. Poor risk analysis can lead to the project failing.
- **Indefinite Loops:** There is a risk of the project continuing in loops without a clear and final endpoint.

# The V-Model

The V-Model is an extension of the Waterfall model, but instead of moving down linearly, it bends upwards after the "coding" phase to form a "V" shape. The key principle is that every phase in the development lifecycle has a corresponding testing phase planned at the same time.

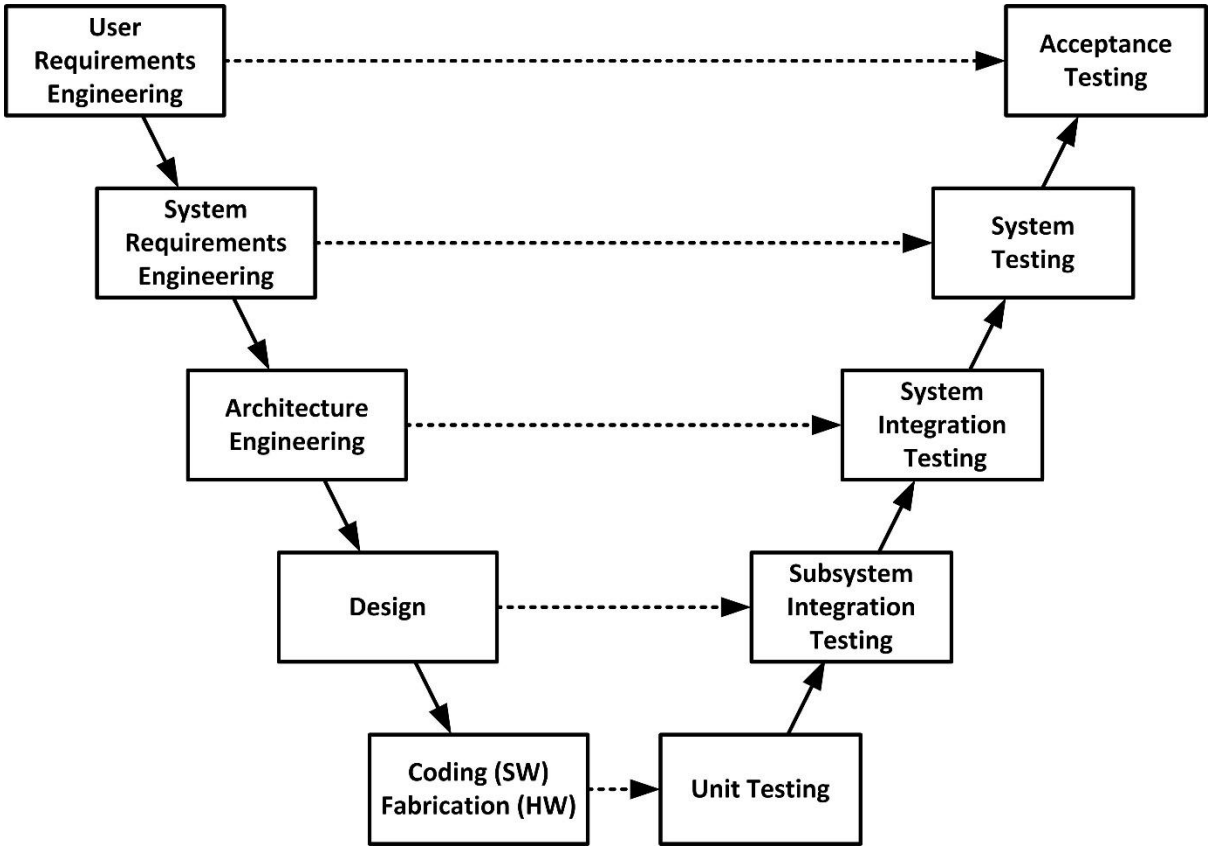


**Figure 5:** The V-Model (Firesmith, D).

## Advantages

- **Highly Disciplined:** Each phase has specific deliverables and a review process which makes it easy to manage.
- **Early Testing:** Early test planning helps developers better understand the requirements and design, reducing the likelihood of introducing defects early on.
- **Clear Structure:** Early testing results in higher-quality and reliable code.
- **Efficient Tracking:** As it is sequential, it allows for accurate progress tracking.

## Disadvantages

- **Inflexible:** If the requirements change mid-development, the entire documentation must be updated.
- **No Early Prototypes:** Working software is not produced until very late in the lifecycle, which means that early feedback is impossible. This makes it not suitable for complex or long-term projects.
- **Late Detection of Issues:** Despite early test planning, actual testing occurs late in the lifecycle.
- **No Iteration:** It is linear, so it does not support changes to previous stages once they are completed.

# Justification: Why Agile and Kanban Were Chosen

For the development of my project, I chose to use Agile as the overall methodology but paired with a Kanban board on GitLab so that I could easily organise and keep track of my work.

I chose to do this as I have used this well previously and I thought that building an application with real-world integrations can be unpredictable, so this methodology would easily be able to adapt to that.

Here is why this combination worked best for my project:

- **Visual Workflow Management:** I maintained a Kanban board in GitLab so that I could visually track all tasks, bugs, and features through stages (e.g. To Do, In Progress, Blocked, Done). When any unexpected issues arose, I immediately logged the tickets as Blockers on the board, prioritised them, and resolved them without bottlenecking any other development tasks.
- **Iterative Development:** Instead of trying to code the entire system at once, the application was built in functional increments. By using the Kanban board to track specific user stories, I focused on delivering working Minimum Viable Products (MVPs) for specific features. For example, I built the checkout system, tested it, then iteratively expanded it later on to include the Google Maps geofencing and Stripe payment integrations.
- **Continuous Refinement:** I carried out testing through the project instead of just a single phase at the end. For example, when testing the delivery zones, the system was calculating the estimated time of arrival for a delivery incorrectly. Because I am using Agile, I could immediately loop back, fix the geocoding issue, and push a patch within the same development lifecycle.

In conclusion, using traditional methodologies like Waterfall would have been too rigid for the project, especially as integrating unfamiliar third-party APIs required a lot of trial and error. The flexibility of Agile, combined with the visual organisation that a Kanban board provides, gave me the structure I needed to build the application successfully.

# References


Atlassian. (n.d.). *Kanban*. [Online] Available at: https://www.atlassian.com/agile/kanban (Accessed 7th March 2026).

Atlassian. (n.d.). *Waterfall Methodology: A Comprehensive Guide.*[Online] Available at: https://www.atlassian.com/agile/project-management/waterfall-methodology (Accessed 7th March 2026).

Atlassian. (n.d.). *What is Agile?* [Online] Available at: https://www.atlassian.com/agile (Accessed 7th March 2026).

Firesmith, D. (2013). *Using V Models for Testing*. [Online] Software Engineering Institute. Available at: https://www.sei.cmu.edu/blog/using-v-models-for-testing/ (Accessed 7th March 2026).

GeeksforGeeks. (2025). *SDLC V-Model – Software Engineering*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering/software-engineering-sdlc-v-model/ (Accessed 7th March 2026).

GeeksforGeeks. (2025). *Why Spiral Model is called Meta Model?* [Online] Available at: https://www.geeksforgeeks.org/software-engineering/why-spiral-model-is-called-meta-model/ (Accessed 7th March 2026).

GeeksforGeeks. (2026). *Spiral Model in Software Engineering*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/ (Accessed 7th March 2026).

Infinity. (n.d.). *Kanban Methodology – A Visual System for Managing Work*. [Online] Available at: https://startinfinity.com/project-management-methodologies/kanban (Accessed 7th March 2026).

Jayathilaka, C. (2020). *Agile Methodology*. [Online] Medium. Available at: https://medium.com/@chathmini96/agile-methodology-30ec4cdf3fc (Accessed 7th March 2026).

TutorialsPoint. (n.d.). *SDLC – Waterfall Model*. [Online] Available at: https://www.tutorialspoint.com/sdlc/sdlc_waterfall_model.htm (Accessed 7th March 2026).